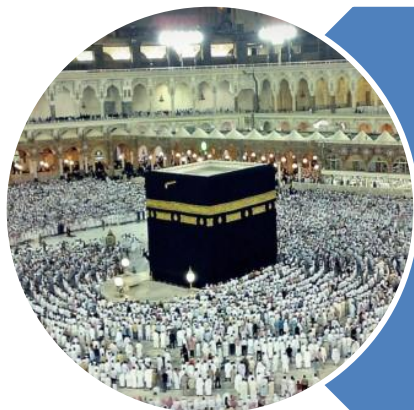


رَبِّ اشْرَحْ لِي صَدْرِي وَيَسِّرْ لِي أَمْرِي وَاحْلُلْ عُقْدَةً مِنْ لِسَانِي يَفْقَهُوا قَوْلِي



O my Lord! Expand for me my chest [with assurance] and ease for me my task and untie the knot from my tongue that they may understand my speech. **(Quran 20 : 25-28)**

Heap

Introduction

C

P

C

S

2

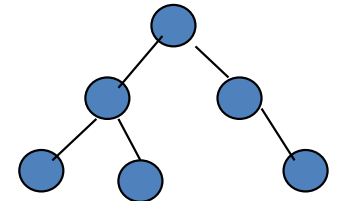
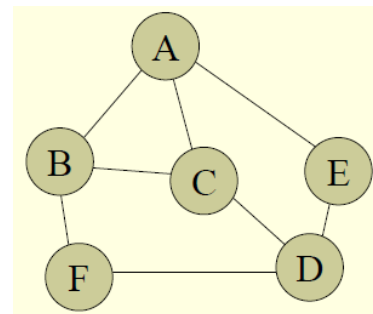
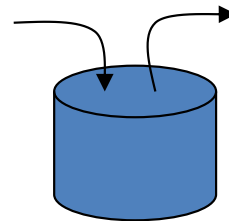
0

4

3

Trees - Introduction

- Arrays
- Linked List
- Stacks
- Queues
- **Tree**
- Graph



Heap - Introduction

- Definition

- A heap is an Abstract Data Type, is a variant of tree data structure

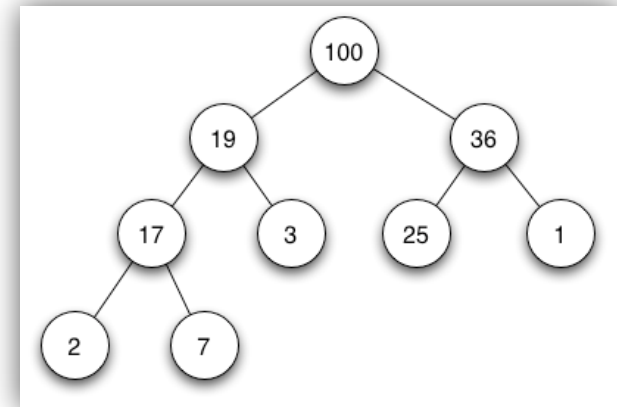
- Heap Properties

- Order / heap Property

- Children **Values are arranged** with respect to root

- Shape Property

- **Complete** Binary Tree



C

P

C

S

2

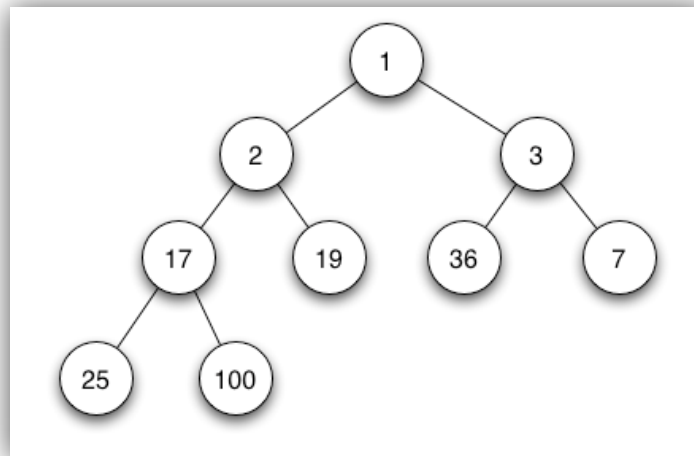
0

4

5

Heap - Types

- Min – Heap
 - If all values stored in the **root** of a given tree is **smaller** than the value stored in the subtrees (**Order / heap Property**)
 - It is Complete Binary Tree (**Shape Property**)



C

P

C

S

2

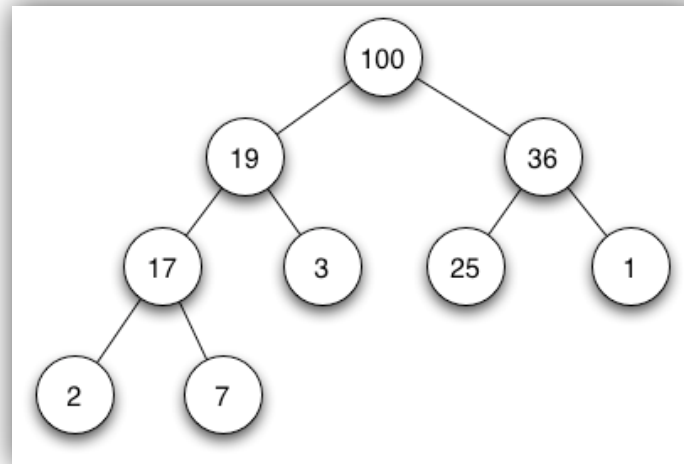
0

4

6

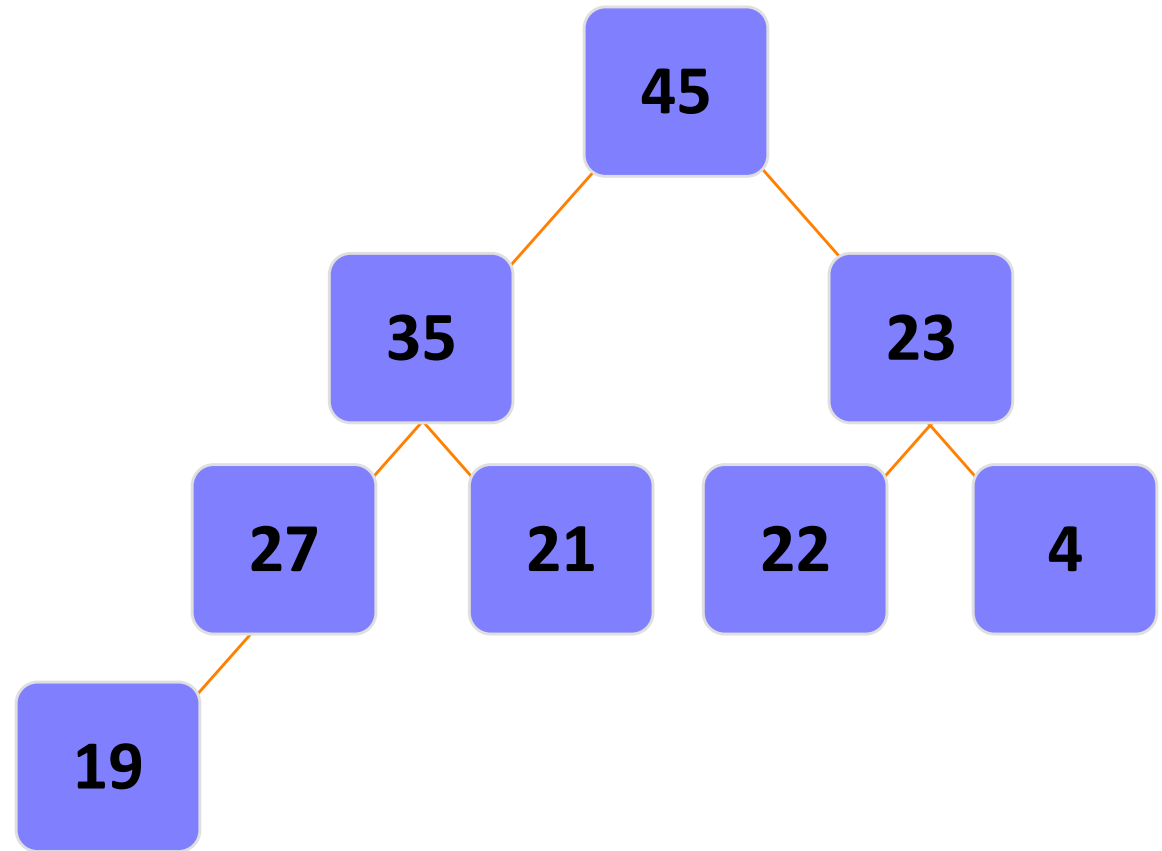
Heap - Types

- Max – Heap
 - If all values stored in the **root** of a given tree is **greater or equal** to the value stored in the subtrees (**Order / heap Property**)
 - It is Complete Binary Tree (**Shape Property**)



Heap – Building Up

- This is called
 - **Max - Heap**



C

P

C

S

2

0

4

Heap – Building Up

- New nodes are always added at the lowest level
 - And are inserted from left to right
- There is no particular relationship among the data items in nodes on any given level
 - Even if the nodes have the same parent
 - Example: the right node does not necessarily have to be larger than the left node (as in BSTs)
- The only ordering property for heaps is the one already defined
 - Root of any given subtree is either largest or smallest element in that tree...either a max-heap or a min-heap

C

P

C

S

2

0

4

Heap – Building Up

- The tree never becomes unbalanced
- A heap is not a sorted structure
 - But it can be regarded as partially ordered
 - Since the minimum / maximum value is always at the root
- A given set of data can be formed into many different heaps
 - Depending on the order in which the data arrives

Heap Operations

C

P

C

S

2

0

4

Heap - Operations

- Adding Node
 - Insertion
- Deleting Node
 - Deletion
- Building Heap
 - Creation

Heap Operations

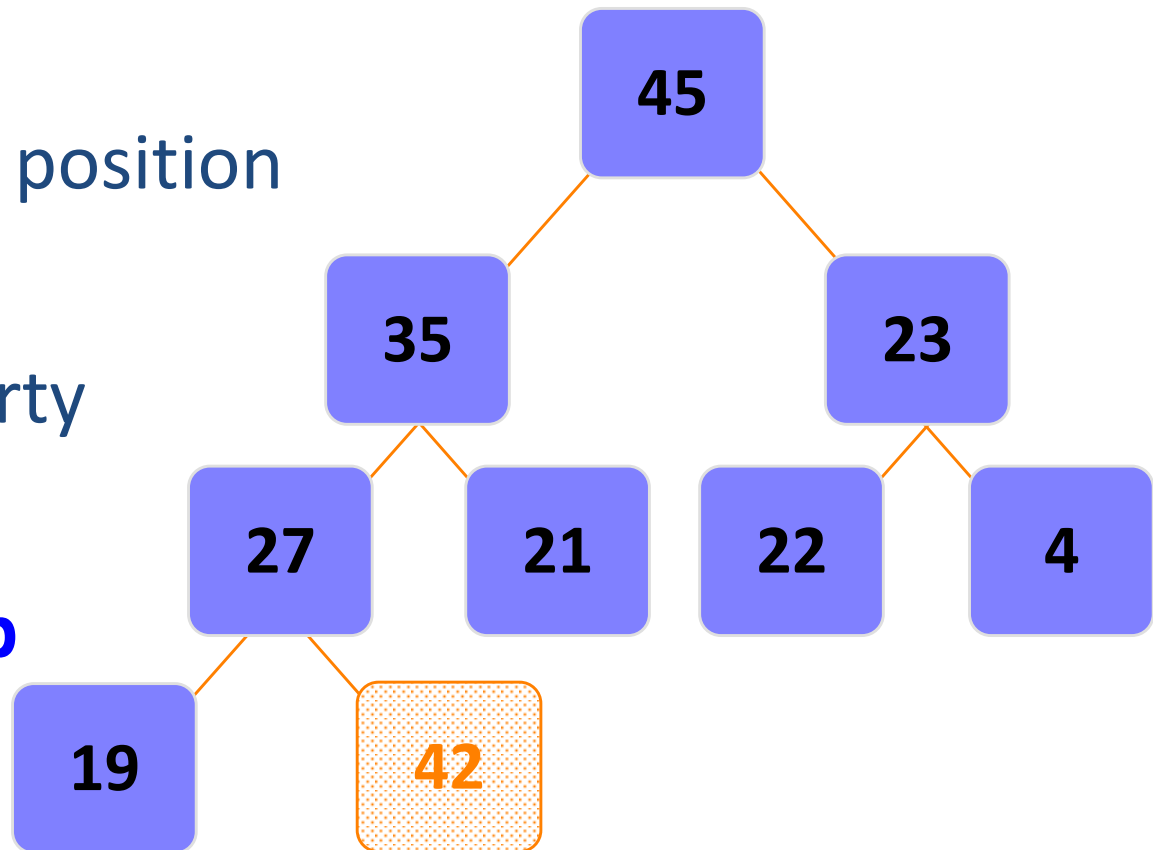
Insertion

Operation - **Insertion**

- Assume the existence of a current heap
- Remember:
 - The binary heap MUST follow the Shape property
 - The tree must be balanced
- Insertions will be made in the next available spot
 - Meaning, at the last level
 - and at the next spot, going from left to right
- But what will most likely happen when you do this?
 - The Heap / Order property will NOT be maintained

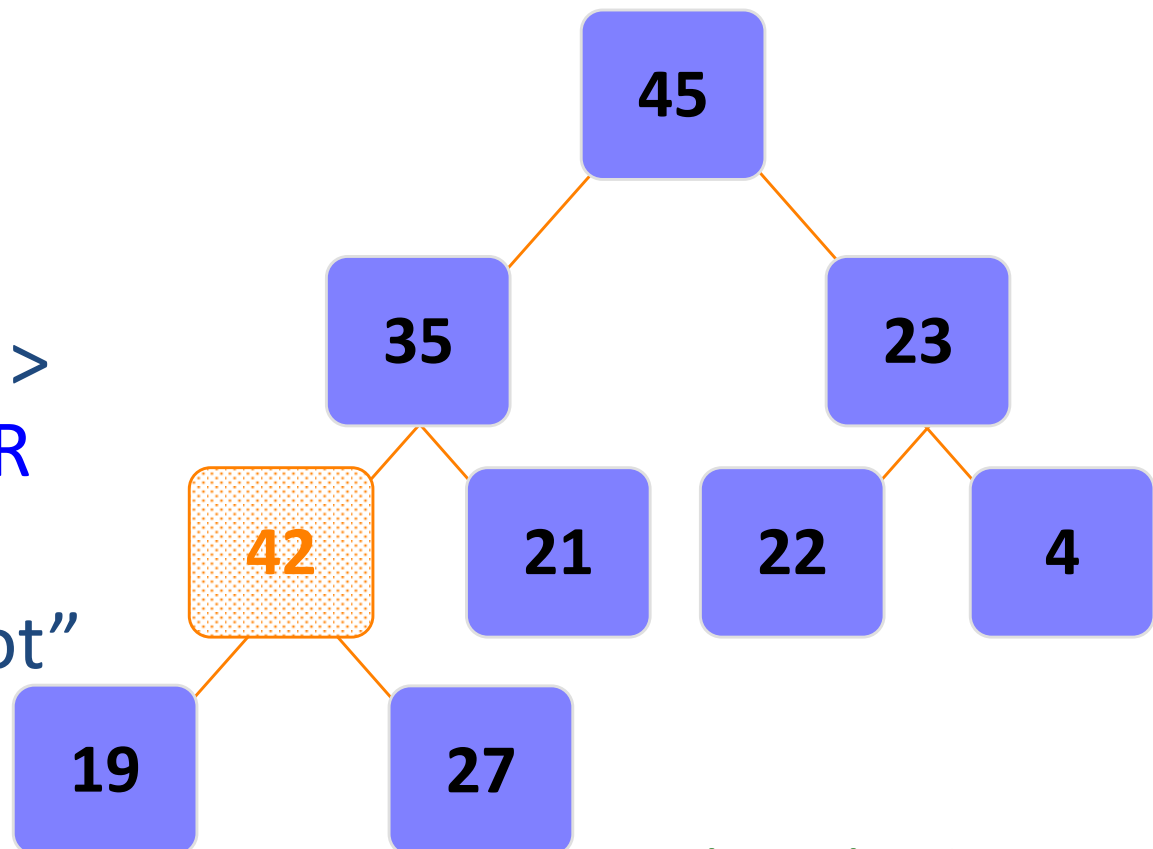
Operation - **Insertion**

- Given Max-Heap
- Add “42”
 - Insert at last position
- It violates
 - Order property
- Fix it
 - **Percolate Up**



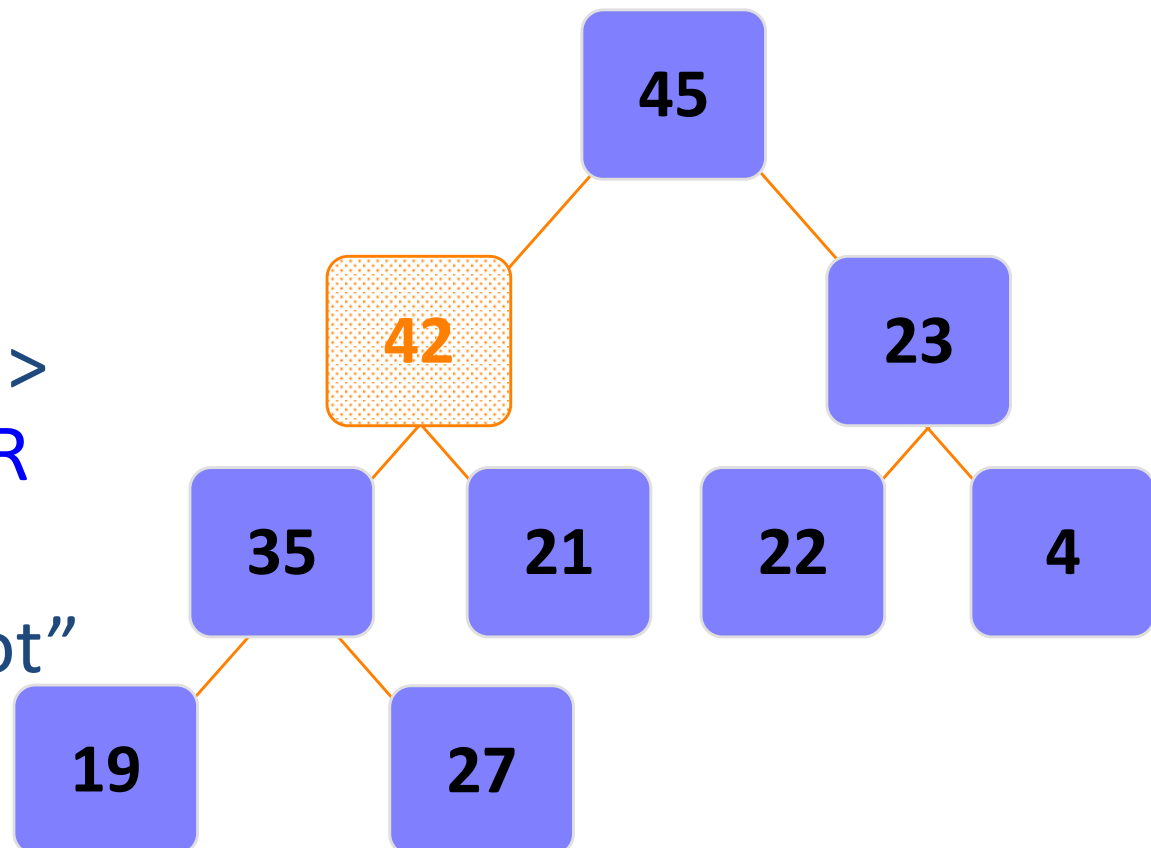
Insertion – Percolate Up

- If new node > parent node
 - Swap them
- Continue till
 - Parent node > new node **OR**
 - New node become “root”



Insertion – Percolate Up

- If new node > parent node
 - Swap them
- Continue till
 - Parent node > new node **OR**
 - New node become "root"



C

P

C

S

2

0

4

Insertion – Analysis

- What is the Big-O running time of insertion into a heap?
- The actual insertion is simply
 - $O(1)$
- Percolate Up
 - Height of tree
 - $O(\log n)$
- Overall **$O(\log n)$**

Heap Operations

Deletion

C

P

C

S

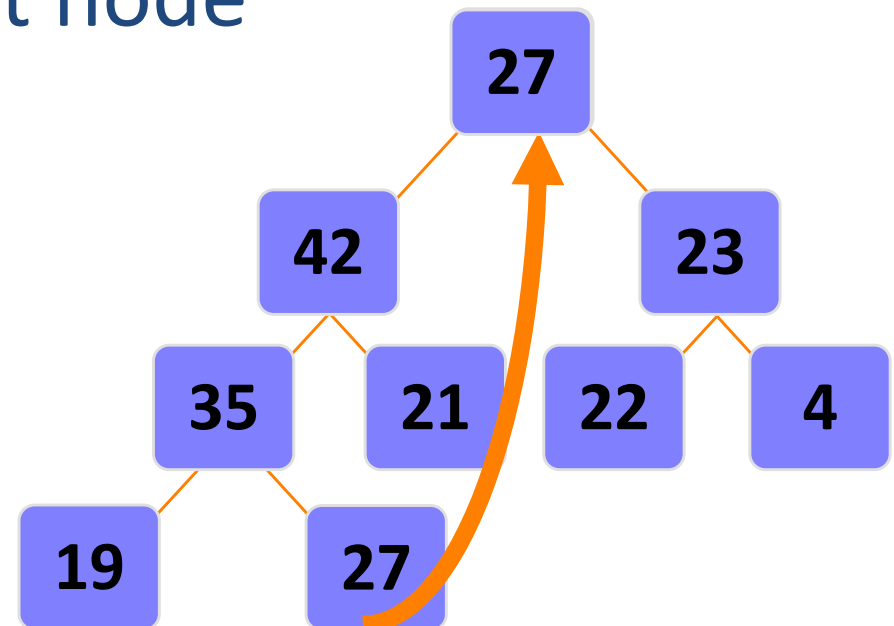
2

0

4

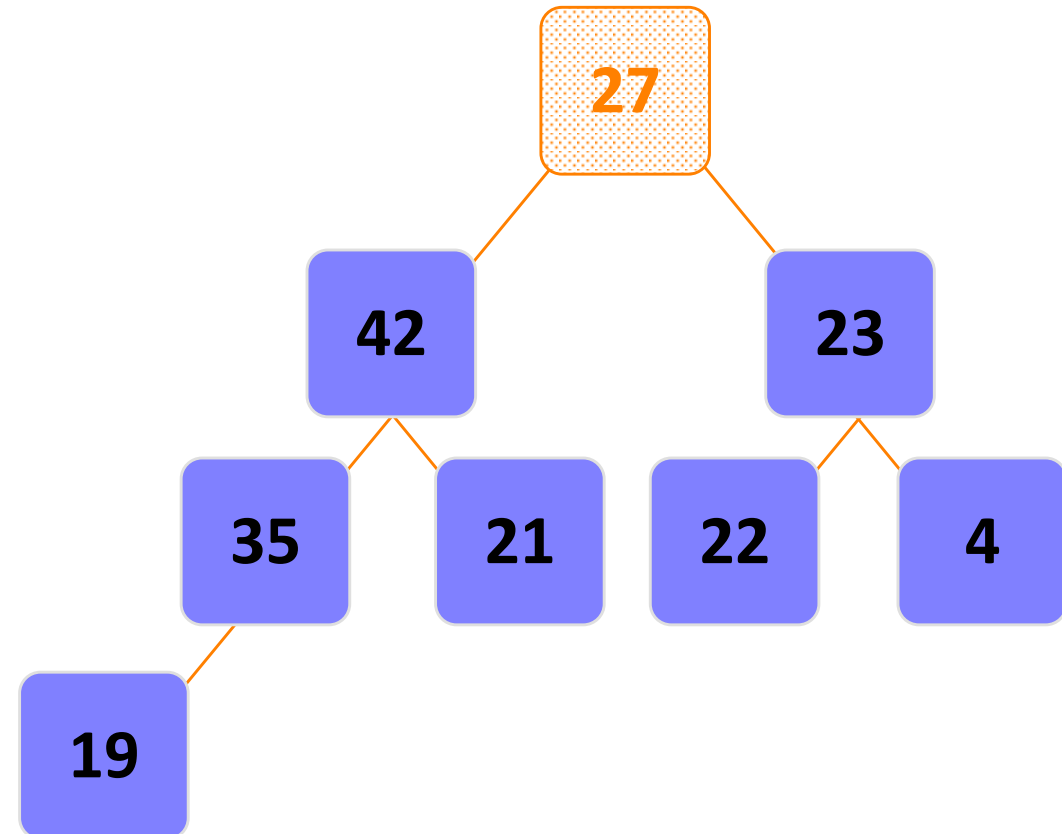
Operation - **Deletion**

- It is similar to STACK deletion
- Only Value at ROOT will be deleted
- Fill the space by last node
- Move downwards
 - **Percolate Down**



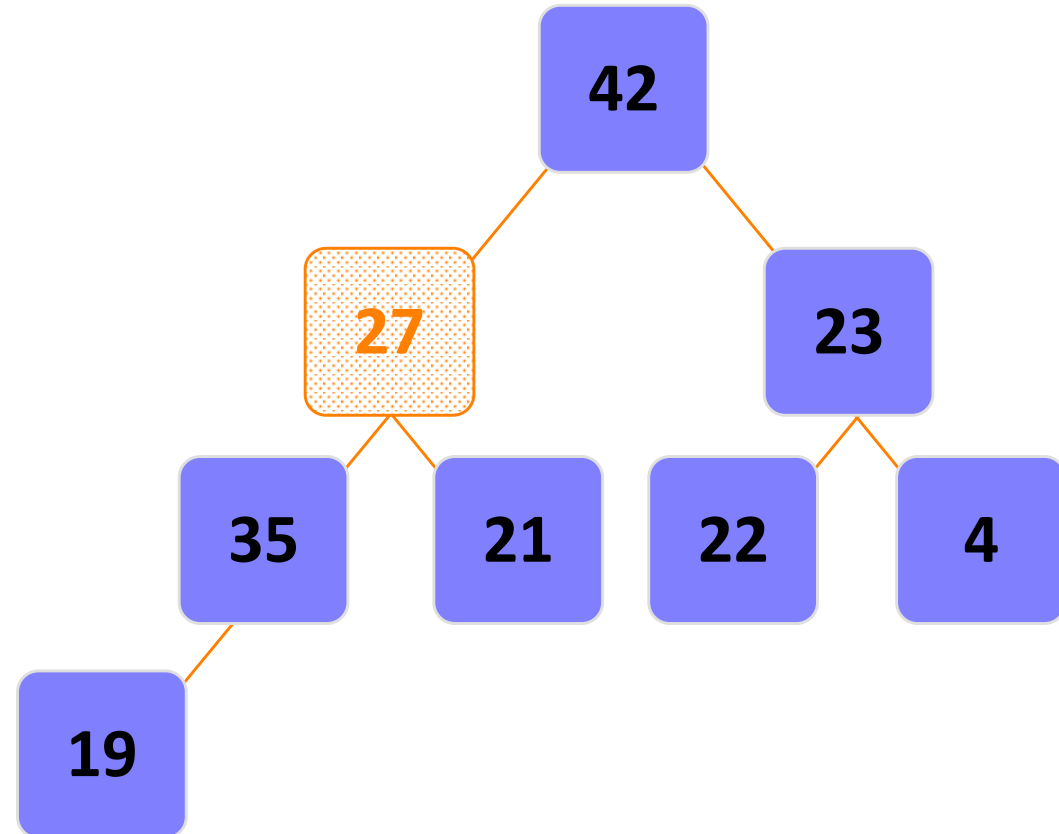
Deletion – Percolate Down

- If new node < children node(s)
 - Swap them
- Continue till
 - new node > Children nodes
OR
 - New node become “leaf”



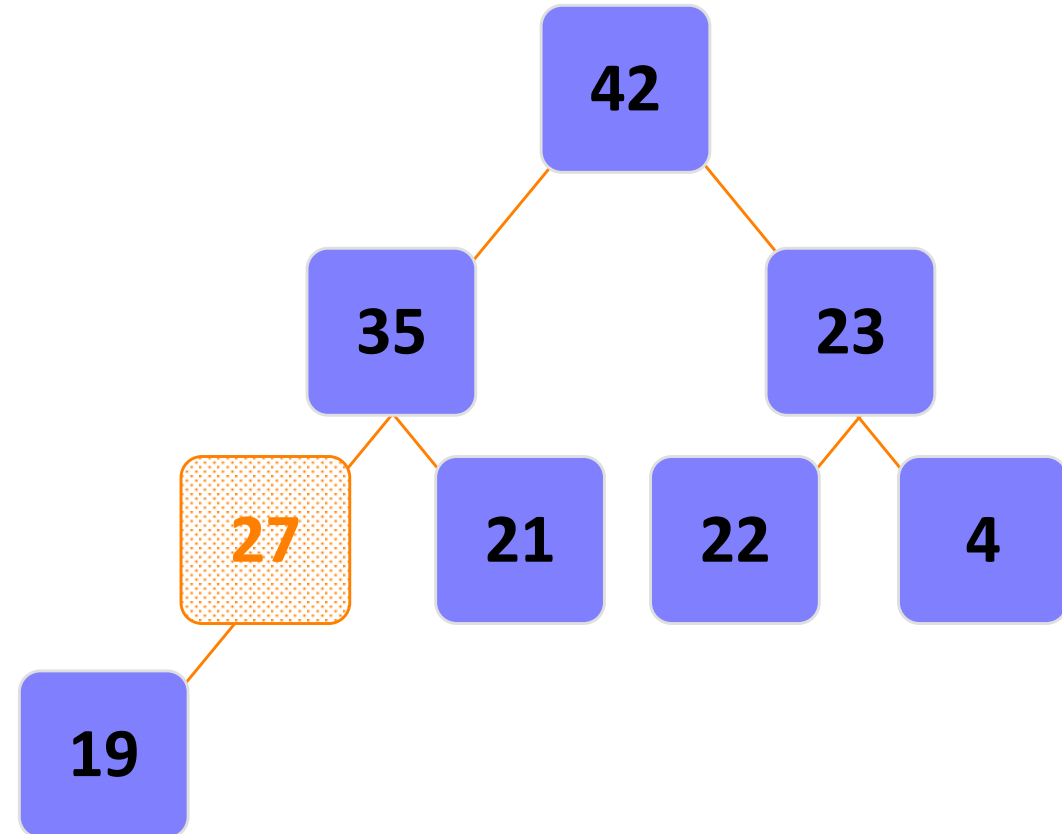
Deletion – Percolate Down

- If new node < children node(s)
 - Swap them
- Continue till
 - new node > Children nodes
OR
 - New node become “leaf”



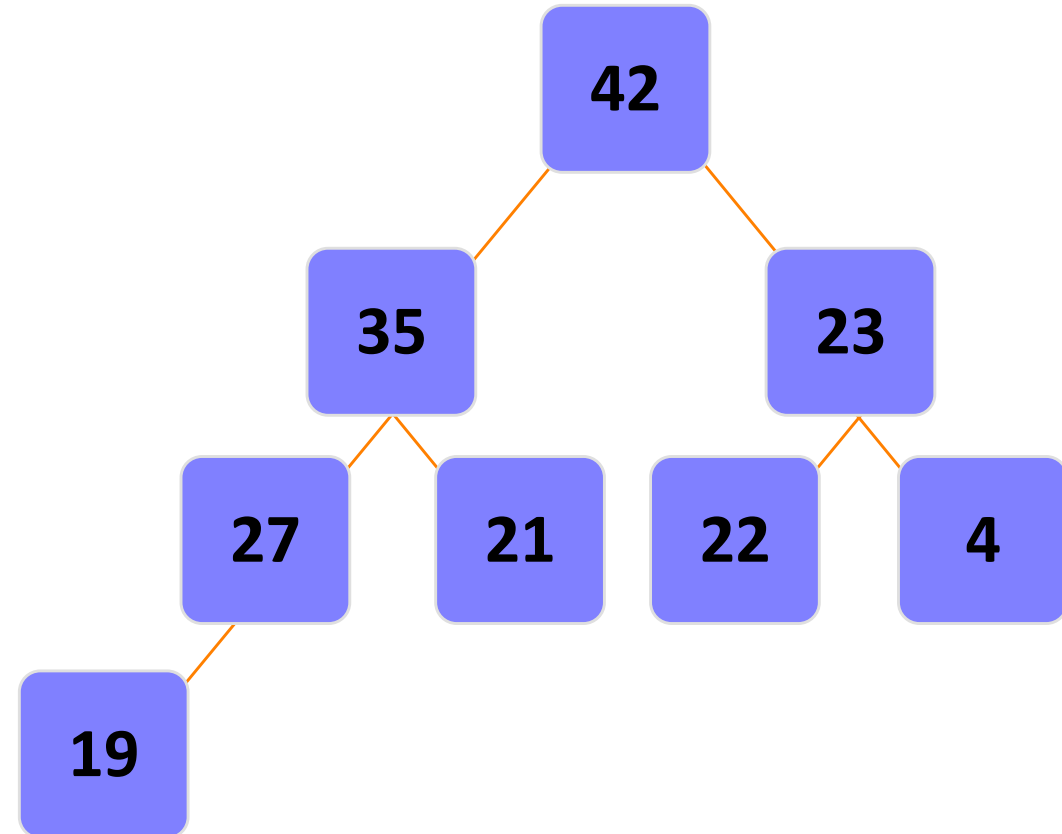
Deletion – Percolate Down

- If new node < children node(s)
 - Swap them
- Continue till
 - new node > Children nodes
OR
 - New node become "leaf"



Deletion – Percolate Down

- If new node < children node(s)
 - Swap them
- Continue till
 - new node > Children nodes
OR
 - New node become “leaf”



C

P

C

S

2

0

4

Deletion – Analysis

- What is the Big-O running time of Deletion from a heap?
- The actual deletion from root
 - $O(1)$
- Percolate Down
 - Height of tree
 - $O(\log n)$
- Overall **$O(\log n)$**

Heap Operations

Creation

Heap – Creation

- Heap is Empty
- First value will make the “root”
- It is really just a series of “insertions”
- Simply insert the n elements into the heap in the order that they arrive (in our case, from left to right)
- Heap is created

C

P

C

S

2

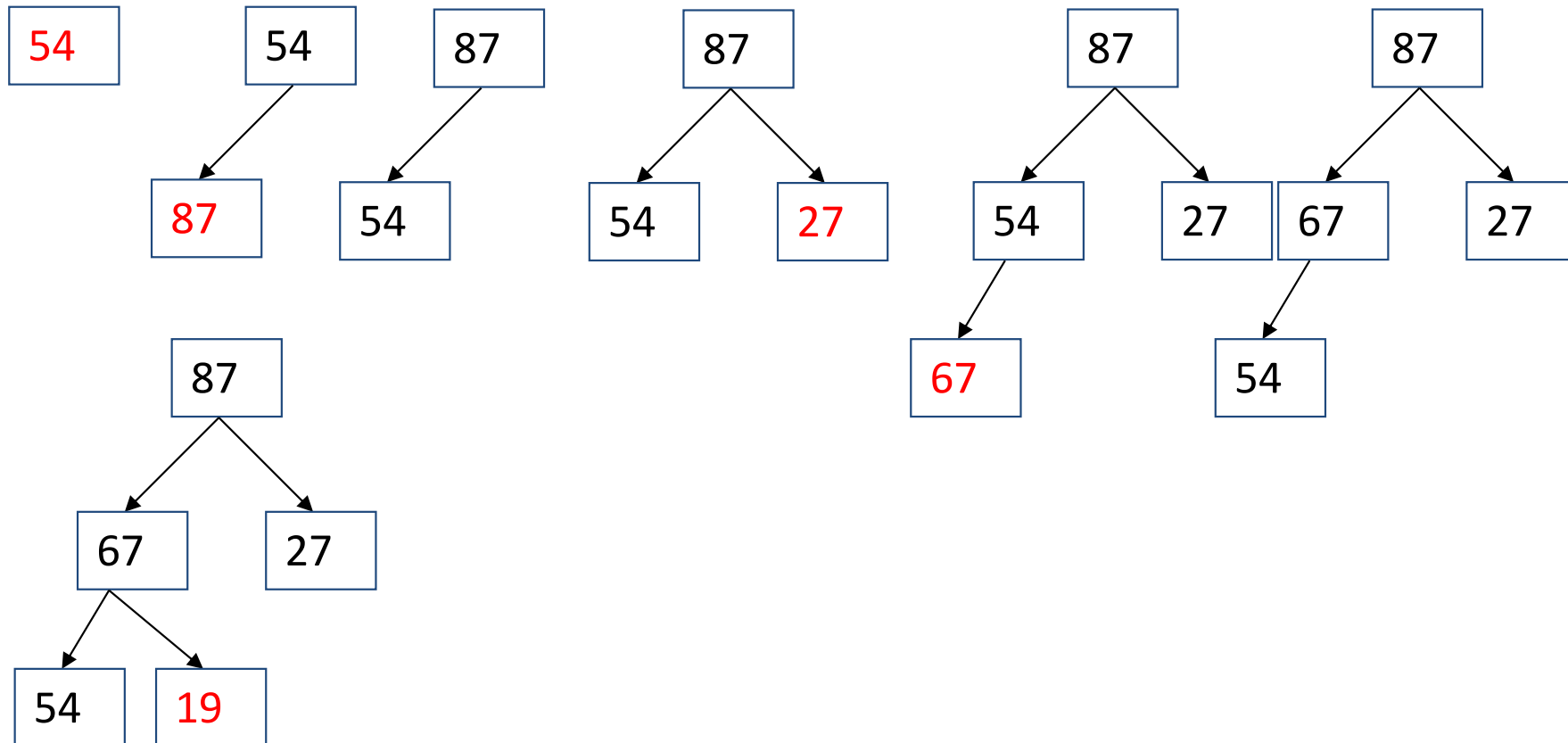
0

4

27

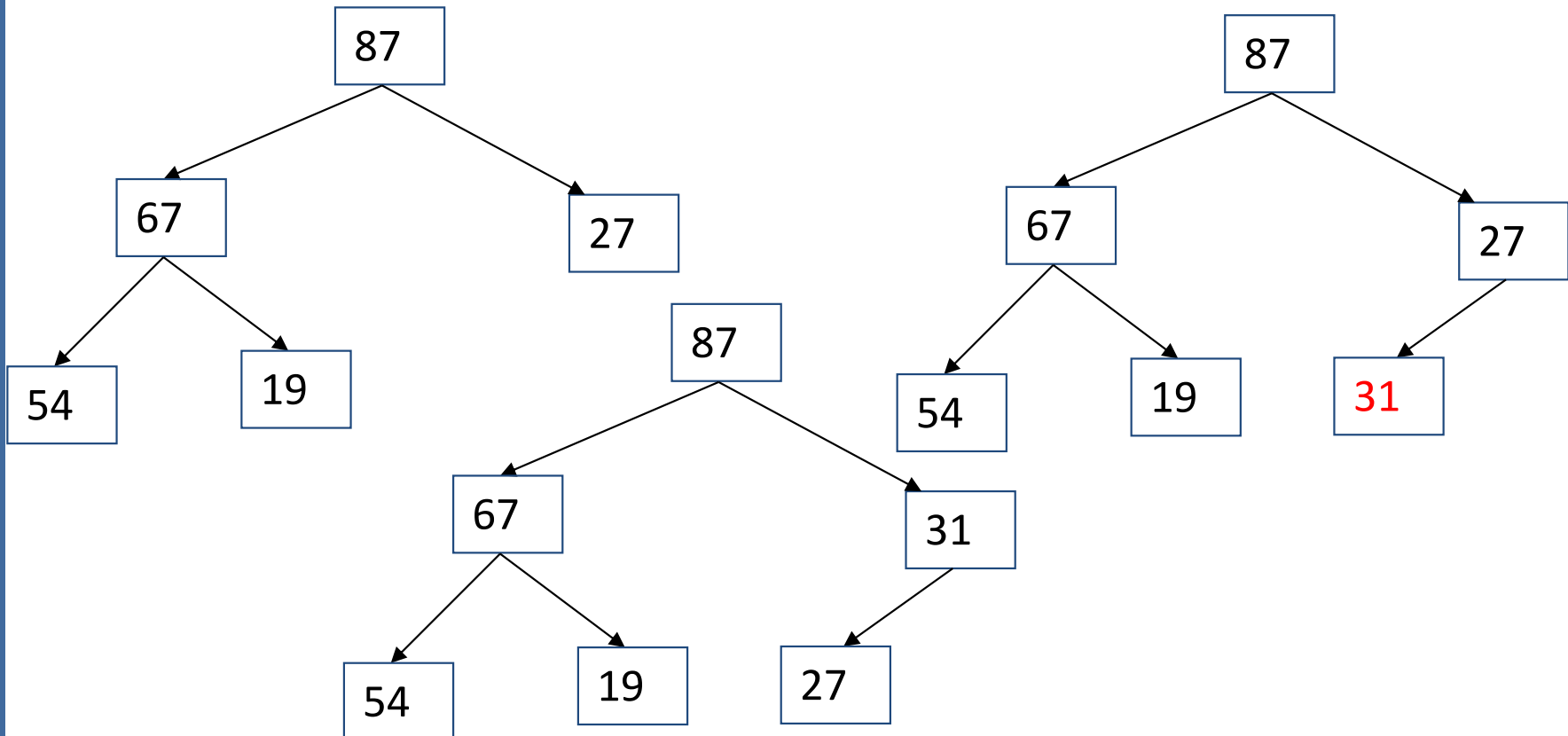
Heap – Creation

- 54, 87, 27, 67, 19, 31, 29, 18, 32, 56, 7, 12, 31



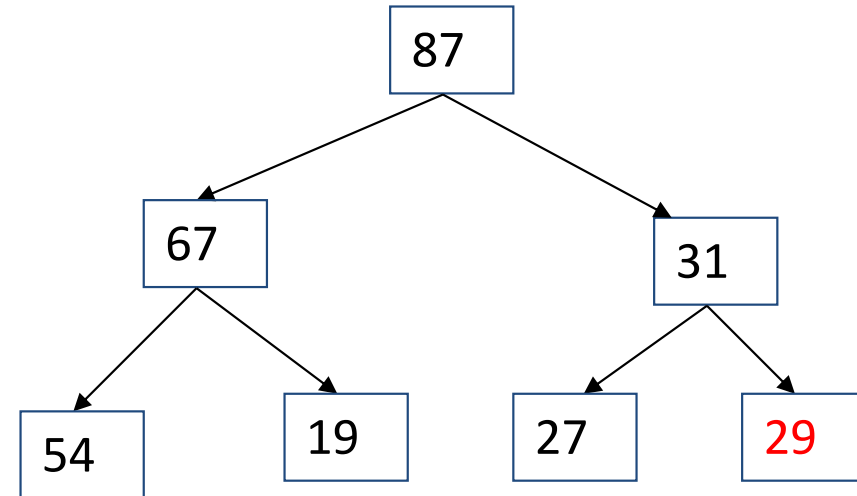
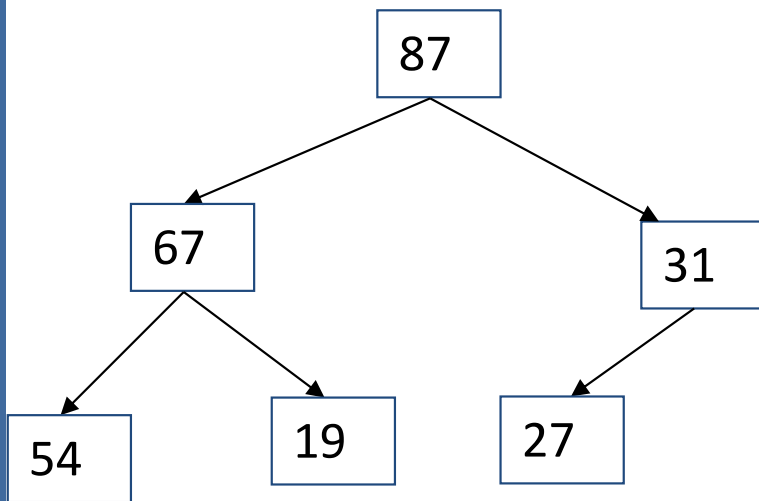
Heap – Creation

- 54, 87, 27, 67, 19, 31, 29, 18, 32, 56, 7, 12, 31



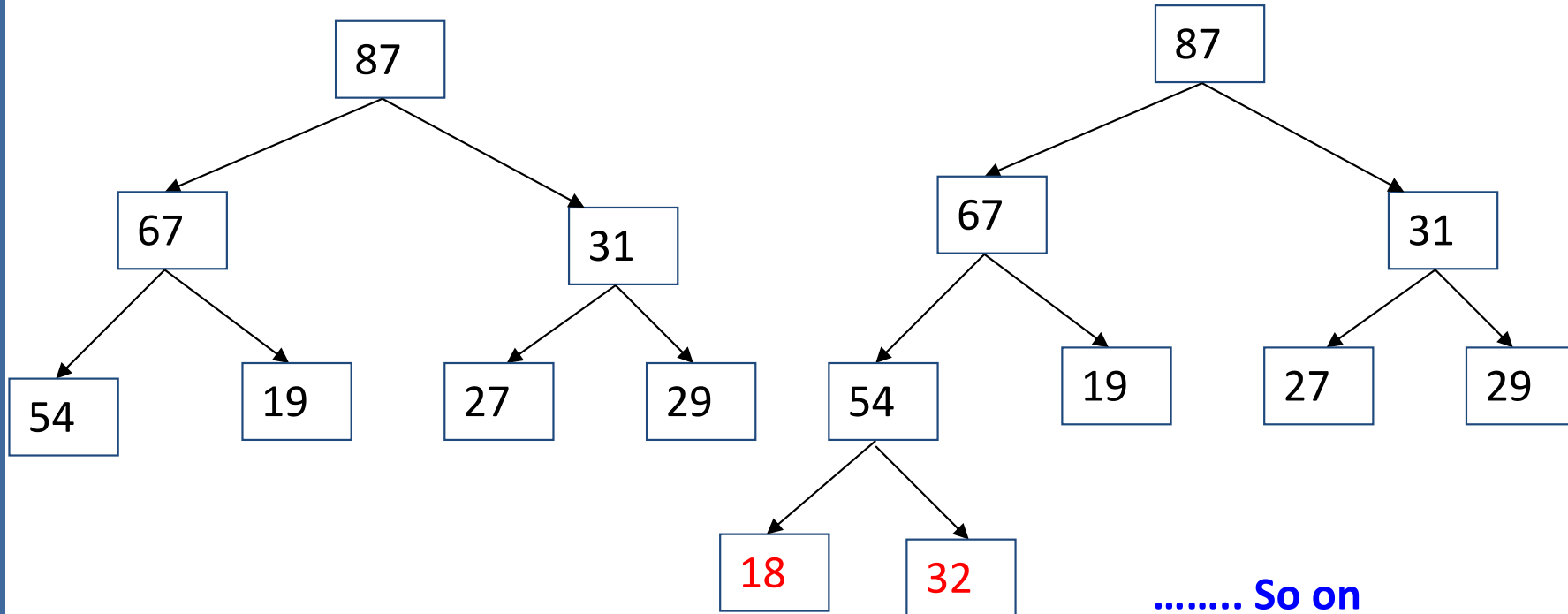
Heap – Creation

- 54, 87, 27, 67, 19, 31, 29, 18, 32, 56, 7, 12, 31



Heap – Creation

- 54, 87, 27, 67, 19, 31, 29, 18, 32, 56, 7, 12, 31



C

P

C

S

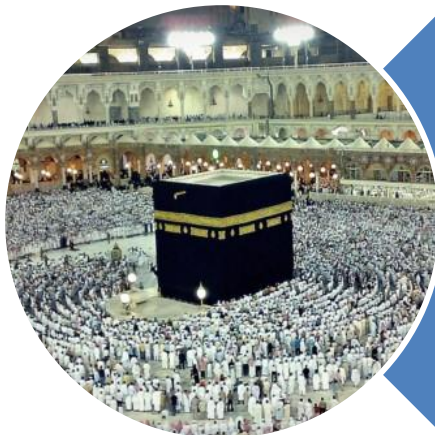
2

0

4

Creation – Analysis

- What is the Big-O running time of building a heap?
- Inserting “one” element
 - $O(\log n)$
- Inserting “n” elements
 - $O(n \log n)$
- Building heap **$O(n \log n)$**



Allah (Alone) is Sufficient for us,
and He is the Best Disposer of
affairs (for us). **(Quran 3 : 173)**